ความสัมพันธ์ระหว่างตาราง (Table Relationships)

แบบ 1:1, 1:N, N:M พร้อมตัวอย่าง SQL และ PHP (mysqli)

ภาพรวม

ประเภทความสัมพันธ์	ตัวอย่าง	จุดสังเกตในโครงสร้าง
1 ต่อ 1 (One-to-One)	users - user_profiles	Foreign Key มี UNIQUE
1 ต่อ กลาย (One-to-Many)	customers - orders	FK อยู่ฝั่ง "กลาย" ไม่ต้อง UNIQUE
กลาย ต่อ กลาย (Many-to-Many)	students - courses	ต้องมีตารางกลาง (Junction Table)

1. ความสัมพันธ์แบบ 1 ต่อ 1 (One-to-One)

ตัวอย่าง: ตาราง users กับ user_profiles — ผู้ใช้ 1 คน มีโปรไฟล์ได้แค่ 1 อันเท่านั้น หัวใจสำคัญคือการใส่ UNIQUE บนคอลัมน์ที่เป็น Foreign Key เพื่อบังคับไม่ให้ซ้ำ

โครงสร้างตาราง (SQL)

```
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL,
    email VARCHAR(100) NOT NULL
);

CREATE TABLE user_profiles (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL UNIQUE,   -- UNIQUE คือหัวใจของ 1:1
    full_name VARCHAR(100),
    phone VARCHAR(20),
    address TEXT,
    FOREIGN KEY (user_id) REFERENCES users(id)
);
```

ตัวอย่างข้อมูล (INSERT)

```
INSERT INTO users (id, username, email) VALUES
(1, 'nattapong', 'nattapong@email.com'),
(2, 'suda_k', 'suda@email.com'),
(3, 'mchai99', 'mchai@email.com');

INSERT INTO user_profiles (id, user_id, full_name, phone, address) VALUES
(1, 1, 'ณัฐพงษ์ ใจดี', '081-234-5678', '123 ถ.สุขุมวิท กรุงเทพฯ'),
(2, 2, 'สุดา คงมั่น', '089-876-5432', '45 ถ.รัชดา กรุงเทพฯ'),
(3, 3, 'มีชัย ตั้งใจ', '062-111-2222', '78 ถ.พระราม 9 กรุงเทพฯ');
```

หมายเหตุ: user_id = 1 จะมีได้แค่ 1 แถวใน user_profiles เท่านั้น เพราะมี UNIQUE constraint คุมอยู่

PHP (mysqli) — ดึงข้อมูล

```
<?php
$conn = new mysqli("localhost", "root", "", "mydb");

$sql = "SELECT u.username, u.email, p.full_name, p.phone
        FROM users u
```

```
        JOIN user_profiles p ON u.id = p.user_id
        WHERE u.id = ?";

$stmt = $conn->prepare($sql);
$stmt->bind_param("i", $userId);
$userId = 1;
$stmt->execute();
$result = $stmt->get_result();
$row = $result->fetch_assoc();

print_r($row);
?>
```

2. ความสัมพันธ์แบบ 1 ต่อ กลาย (One-to-Many)

ตัวอย่าง: ตาราง customers กับ orders — ลูกค้า 1 คน สั่งซื้อได้หลายออเดอร์ โดย Foreign Key จะอยู่ฝั่ง "กลาย" (orders) และ ไม่ต้องมี UNIQUE เพราะค่าซ้ำกันได้

โครงสร้างตาราง (SQL)

```
CREATE TABLE customers (
   id INT AUTO_INCREMENT PRIMARY KEY,
   name VARCHAR(100) NOT NULL
);

CREATE TABLE orders (
   id INT AUTO_INCREMENT PRIMARY KEY,
   customer_id INT NOT NULL,      -- ไม่ต้อง UNIQUE เพราะซ้ำได้
   order_date DATE,
   total DECIMAL(10,2),
   FOREIGN KEY (customer_id) REFERENCES customers(id)
);
```

ตัวอย่างข้อมูล (INSERT)

```
INSERT INTO customers (id, name) VALUES
(1, 'ร้านค้าเอบีซี'),
(2, 'คุณสมชาย ใจงาม'),
(3, 'บริษัท XYZ จำกัด');

INSERT INTO orders (id, customer_id, order_date, total) VALUES
(1, 1, '2025-01-05', 1500.00),
(2, 1, '2025-01-12', 2300.50),
(3, 1, '2025-02-01', 890.00),
(4, 2, '2025-01-20', 450.00),
(5, 3, '2025-01-25', 12000.00),
(6, 3, '2025-02-03', 8750.00);
```

หมายเหตุ: customer_id = 1 (ร้านค้าเอบีซี) มีออเดอร์ 3 รายการ นี่คือลักษณะของ "1 ต่อกลาย"

PHP (mysqli) — ดึงออเดอร์ทั้งหมดของลูกค้า 1 คน

```
<?php
$conn = new mysqli("localhost", "root", "", "mydb");

$sql = "SELECT o.id, o.order_date, o.total
        FROM orders o
        WHERE o.customer_id = ?";

$stmt = $conn->prepare($sql);
$stmt->bind_param("i", $customerId);
$customerId = 5;
$stmt->execute();
$result = $stmt->get_result();

while ($row = $result->fetch_assoc()) {
   echo "Order #{$row['id']} - {$row['order_date']} - {$row['total']} บาท\n";
}
?>
```

3. ความสัมพันธ์แบบ หลายต่อหลาย (Many-to-Many)

ตัวอย่าง: ตาราง students กับ courses — นักเรียนคนหนึ่งเรียนได้หลายวิชา และวิชาหนึ่งก็มีนักเรียนหลายคน ต้องมี ตารางกลาง (Junction / Pivot Table) เพื่อเก็บคู่ความสัมพันธ์

โครงสร้างตาราง (SQL)

```
CREATE TABLE students (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL
);

CREATE TABLE courses (
    id INT AUTO_INCREMENT PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL
);

-- ตารางกลาง (Pivot Table)
CREATE TABLE student_course (
    student_id INT NOT NULL,
    course_id INT NOT NULL,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES students(id),
    FOREIGN KEY (course_id) REFERENCES courses(id)
);
```

ตัวอย่างข้อมูล (INSERT)

```
INSERT INTO students (id, name) VALUES
(1, 'สมหญิง รักเรียน'),
(2, 'วิชัย ขยันดี'),
(3, 'อรทัย ตั้งใจ');

INSERT INTO courses (id, course_name) VALUES
(1, 'คณิตศาสตร์'),
(2, 'วิทยาศาสตร์'),
(3, 'ภาษาอังกฤษ'),
(4, 'คอมพิวเตอร์');

INSERT INTO student_course (student_id, course_id) VALUES
(1, 1),  -- สมหญิง เรียน คณิตศาสตร์
(1, 3),  -- สมหญิง เรียน ภาษาอังกฤษ
(1, 4),  -- สมหญิง เรียน คอมพิวเตอร์
(2, 2),  -- วิชัย เรียน วิทยาศาสตร์
(2, 4),  -- วิชัย เรียน คอมพิวเตอร์
(3, 1),  -- อรทัย เรียน คณิตศาสตร์
(3, 2),  -- อรทัย เรียน วิทยาศาสตร์
(3, 3);  -- อรทัย เรียน ภาษาอังกฤษ
```

ผลลัพธ์เมื่อ JOIN ดู

นักเรียน	วิชาที่ลงทะเบียน
สมหญิง	คณิตศาสตร์, ภาษาอังกฤษ, คอมพิวเตอร์
วิชัย	วิทยาศาสตร์, คอมพิวเตอร์
อรทัย	คณิตศาสตร์, วิทยาศาสตร์, ภาษาอังกฤษ

สังเกตว่า "คอมพิวเตอร์" มีนักเรียนเรียน 2 คน (สมหญิง, วิชัย) และ "สมหญิง" ก็เรียนได้หลายวิชา — นี่คือลักษณะของความสัมพันธ์แบบ "หลายต่อหลาย" ตัวจริง

PHP (mysqli) — ดึงวิชาทั้งหมดที่นักเรียนคนหนึ่งลงทะเบียน

```php
<?php
$conn = new mysqli("localhost", "root", "", "mydb");

$sql = "SELECT c.course_name
        FROM courses c
        JOIN student_course sc ON c.id = sc.course_id
        WHERE sc.student_id = ?";

$stmt = $conn->prepare($sql);
$stmt->bind_param("i", $studentId);
$studentId = 3;
$stmt->execute();
$result = $stmt->get_result();

while ($row = $result->fetch_assoc()) {
    echo $row['course_name'] . "\n";
}
?>
```

สรุปวิธีจำง่ายๆ

แบบ	ตัวอย่าง	จุดสังเกตในโครงสร้าง
1:1	user - profile	FK มี UNIQUE
1:กลาย	customer - orders	FK อยู่ฝั่ง "กลาย" ไม่ต้อง unique
กลาย:กลาย	student - course	ต้องมีตารางกลาง (junction table)